OpKAN: Constraining Neural Option Pricing with Physics, Regime Awareness, and Auditable Language Control

OpKAN Engineering Team
github.com/RicardoLaMo/OpKAN

March 2026

Abstract

Options pricing at a modern trading desk is simultaneously a quantitative problem — produce prices that satisfy no-arbitrage conditions and financial boundary structure — and an informational problem — incorporate unstructured qualitative signals (macro news, research narratives, policy commentary) into the pricing model rapidly and without manual re-calibration. Classical approaches address the first problem well but not the second. Machine learning approaches ingest structured data effectively but have no natural channel for unstructured information.

We present **OpKAN**, an architecture that addresses both problems through three complementary layers. A *physics-informed Kolmogorov-Arnold network* (PI-KAN) solves the Heston stochastic volatility partial differential equation over randomly sampled collocation points, enforcing PDE-level financial constraints via the training objective rather than by architectural choice. A *Hidden Markov regime detector* provides a temporally stable, look-ahead-bias-free market state signal that conditions how the pricing surface should be interpreted. A *topology management agent* — a dual-process large language model (LLM) system we call the *semantic compiler* — reads both the regime signal and the learned KAN topology, then issues typed, validated commands that modify the network's structure in real time.

The central design insight is a strict division of labor: the LLM does not price options; it translates unstructured text into a finite, Pydantic-validated domain-specific language (DSL) that drives the quantitative backend. We evaluate this architecture on five protocol benchmark tiers, pricing accuracy benchmarks across three market regimes, and an engineering throughput test on NVIDIA H200 hardware. Our preliminary findings establish that the LLM can compile market-narrative inputs into valid, executable DSL with 1.00 execution and intent-match rates on curated prompts, declining to 0.75 semantic agreement on harder document-grounded benchmarks — a gap that illuminates a concrete normalization research problem. Quantitatively, a regime-conditioned IV-residual hybrid achieves the lowest overall replay price MAE (0.9317) with particular strength in transition and stress regimes, while all five financial monotonicity checks pass only when an explicit semantic controller is layered over the raw learned model. Engineering throughput reaches **26,300** samples/sec on H200 with zero GPU blocking from the asynchronous LLM decision cycle.

These results are exploratory: they demonstrate architectural viability and raise specific, tractable open questions rather than establishing final solutions.

Contents

1 Introduction **4**
1.1 Two Problems That Are Almost Always Treated as One 4
1.2 The OpKAN Approach 4
1.3 Contributions 5

1.4 Scope and Positioning . . . 5
1.5 Paper Organization . . . 5

2 Problem Formulation **6**
2.1 The Heston PDE and the C^2 Constraint . . . 6
2.2 The Regime Problem: Why a Single Learned Model Is Insufficient . . . 6
2.3 The Narrative Problem: Why Structured Models Cannot Self-Update . . . 6

3 Architecture **7**
3.1 Design Philosophy: One Problem per Layer . . . 7
3.2 Layer 1: The Physics-Informed KAN . . . 8
3.2.1 Why KAN Over MLP for Heston . . . 8
3.2.2 B-Spline Edge Activations . . . 8
3.2.3 Model Architecture and Training Objective . . . 8
3.3 Layer 2: Hidden Markov Regime Detector . . . 9
3.3.1 Why a Separate Regime Layer . . . 9
3.3.2 Features and Walk-Forward Inference . . . 9
3.4 Layer 3: The Topology Management Agent . . . 10
3.4.1 The Core Idea: LLM as Semantic Compiler . . . 10
3.4.2 Two-Speed Architecture . . . 10
3.4.3 The Domain-Specific Language . . . 11
3.5 End-to-End Coordination . . . 11

4 Correctness and Security **11**
4.1 The Code Injection Problem . . . 12
4.2 AST Allowlist Validation . . . 12
4.3 Correct Pruning Semantics . . . 13
4.4 Atomic Mutation Rollback . . . 13

5 Related Work **14**
5.1 Physics-Informed Learning for PDEs . . . 14
5.2 Kolmogorov-Arnold Networks . . . 14
5.3 LLM Tool Use and Structured Output . . . 14
5.4 Market Regime Detection . . . 14
5.5 Dual-Process AI Architectures . . . 15

6 Experimental Investigation **15**
6.1 Research Questions . . . 15
6.2 Experimental Setup . . . 15
6.3 Protocol Benchmark Tiers . . . 16

7 Results and Findings **16**
7.1 RQ1 – Protocol Validity . . . 16
7.2 RQ2 – Paraphrase Stability . . . 17
7.3 RQ3 – Normalization Gap . . . 17
7.4 RQ4 – Regime Coherence . . . 18
7.5 RQ5 – Pricing Performance . . . 18
7.6 RQ6 – Engineering Performance . . . 19

8 Discussion **20**
8.1 What the Architecture Gets Right . . . 20
8.2 The Monotonicity Gap: A Design Question, Not a Model Failure . . . 20
8.3 The Normalization Gap: A Contract Problem . . . 21

8.4 Cross-Asset Transfer: What Domain Specificity Means 21

9 Limitations **21**

10 Future Directions **22**

11 Conclusion **23**

1 Introduction

1.1 Two Problems That Are Almost Always Treated as One

An options desk pricing a call on silver futures at 9:42 on a morning when the Federal Reserve has just announced a larger-than-expected rate hold faces two problems that are structurally different but temporally inseparable.

The *quantitative problem*: given current spot price S, instantaneous variance v, and time-to-expiry t, what option price $V(S, v, t)$ is consistent with the no-arbitrage dynamics of the Heston model, the terminal payoff at expiry, and the boundary behavior as $S \to 0$ and $S \to \infty$?

The *informational problem*: the rate decision is not a number. It is a paragraph. The relevant implication — stress, transition, or diffusion regime? — is a semantic judgment that must be translated into a volatility surface adjustment within seconds, before any market participant who makes this translation more slowly has already traded.

Classical quantitative finance solves the first problem with high fidelity. Black-Scholes [Black and Scholes, 1973], Merton [Merton, 1973], and the Heston stochastic volatility model [Heston, 1993] each provide a mathematically principled PDE with consistent Greeks and well-defined boundary conditions. But these formulas adapt to new regime information only through manual parameter recalibration — a process that takes minutes and requires an analyst's judgment.

Machine learning addresses the second problem better. Neural surface models [Hutchinson et al., 1994] can embed unstructured features through learned representations. Physics-informed neural networks (PINNs) [Raissi et al., 2019] enforce the PDE as a soft constraint during training, combining data-driven flexibility with structural discipline. But existing PINN architectures typically use standard multi-layer perceptrons (MLPs), whose ReLU activations have *zero second derivatives* — a critical deficiency for Heston's cross-derivative term $\partial^2 V/\partial S \partial v$, which requires C^2 continuous activations for stable evaluation.

More fundamentally, no existing architecture provides a principled, auditable channel for converting unstructured narrative text into quantitative topology updates at latencies compatible with real-time trading.

1.2 The OpKAN Approach

OpKAN proposes that the informational problem and the quantitative problem can be solved jointly through a strict *separation of concerns* across three layers, each optimized for what it does well.

Layer 1 — The Physics Layer. A Kolmogorov-Arnold network [Liu et al., 2024] trained with a Heston PDE residual loss replaces the MLP backbone of conventional PINNs. KAN edge activations are parameterized as cubic B-splines, which are genuinely C^2 continuous across knot boundaries and therefore stable under second-order autograd. The resulting model is called a *PI-KAN* (Physics-Informed KAN).

Layer 2 — The Regime Layer. A Gaussian Hidden Markov model runs walk-forward inference over five market microstructure features and emits a temporally stable, permutation-normalized regime label (low-volatility, high-volatility, or stress). This label functions as a conditioning signal that tells downstream components which market environment the current pricing should reflect.

Layer 3 — The Semantic Compiler Layer. A two-speed LLM agent reads the regime signal and the learned KAN topology, then issues typed mutation commands that modify the network's

edge activations in real time. Critically, the LLM's output is constrained to a finite, Pydantic-validated domain-specific language (DSL). The LLM never produces a price; it produces a structured command that a specialist quantitative backend executes. This design makes every agent action auditable, replayable, and revocable.

Together, these three layers form OpKAN — a regime-aware, narrative-responsive option pricing system where quantitative discipline and linguistic flexibility are composable rather than opposed.

1.3 Contributions

This paper makes the following contributions:

1. **PI-KAN for Heston pricing.** A complete PI-KAN implementation using genuine Cox–de Boor cubic B-splines, replacing placeholder activations in prior implementations, with C^2 continuity verified by second-order autograd on the Heston cross-derivative term.
2. **The semantic compiler design pattern.** A formal specification of the LLM-as-compiler architecture: a Pydantic DSL that makes the LLM's output typed, validated, and finite in vocabulary, with provable code-injection protection via AST allowlist analysis.
3. **A two-speed topology management agent.** A dual-process agent architecture that separates high-frequency reflexive edge maintenance (pruning, learning-rate adjustment) from low-frequency strategic topology surgery (symbolic replacement, regime diagnosis), so that neither process blocks the other or the GPU training loop.
4. **Atomic mutation rollback.** A snapshot-and-restore mechanism that prevents partially-mutated model state by treating each mutation batch as a transactional unit.
5. **A tiered evaluation framework.** Five protocol benchmark tiers and two pricing benchmark tiers that separately assess the LLM compilation layer, the regime control layer, and the quantitative pricing layer — exposing the specific boundaries of current competence.
6. **An honest gap analysis.** Explicit identification of what the current architecture does not solve: intrinsic monotonicity, adversarial document robustness, and full human-analyst normalization.

1.4 Scope and Positioning

OpKAN is not a pricing formula and does not claim to outperform all existing pricing models in all conditions. Its specific claim is narrower: *when regime interpretation from unstructured narrative is part of the pricing problem, an architecture that combines a physics-grounded learned model with an auditable language control layer is more effective than either component alone.* The cross-asset experiments in Section 7 show precisely where this claim holds and where it does not.

1.5 Paper Organization

Section 2 develops the problem formulation. Section 3 describes the architecture with explicit design rationale. Section 4 addresses correctness and security. Section 5 situates the work in the literature. Section 6 presents the experimental design and research questions. Section 7 reports findings. Section 8 interprets them and raises open questions. Section 9 states scope boundaries. Section 10 outlines future directions. Section 11 concludes.

2 Problem Formulation

Before describing the architecture, we formalize the constraints that any solution must satisfy. This section establishes three requirements that jointly motivate the design choices in Section 3.

2.1 The Heston PDE and the C^2 Constraint

The Heston stochastic volatility model [Heston, 1993] describes risk-neutral asset dynamics as a joint process for spot price S_t and instantaneous variance v_t:

$$dS_t = rS_t\,dt + \sqrt{v_t}S_t\,dW_t^S \tag{1}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \sigma\sqrt{v_t}\,dW_t^v \tag{2}$$

where κ is mean-reversion speed, θ is the long-run variance, σ is volatility-of-volatility, and $\mathrm{Corr}(dW^S, dW^v) = \rho\,dt$ captures the leverage effect. By Itô's lemma, the fair value $V(S, v, t)$ of a European call satisfies the Heston PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}vS^2\frac{\partial^2 V}{\partial S^2} + \rho\sigma vS\frac{\partial^2 V}{\partial S\partial v} + \frac{1}{2}\sigma^2 v\frac{\partial^2 V}{\partial v^2} + rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV = 0 \tag{3}$$

supplemented by boundary conditions:

$$V(S, v, T) = \max(S - K, 0) \quad \text{(terminal payoff)} \tag{4}$$

$$V(0, v, t) = 0 \quad \text{(zero spot)} \tag{5}$$

$$V(S, v, t) \approx S - Ke^{-r(T-t)} \quad \text{as } S \to \infty \tag{6}$$

The cross-derivative term $\rho\sigma vS\,\partial^2 V/\partial S\partial v$ is the key constraint for architecture design. Evaluating this term via automatic differentiation requires the network activation functions to be *at least C^2 continuous*: once-differentiable activations (such as ReLU or leaky ReLU) produce zero second derivatives and make this term numerically degenerate. This rules out standard MLP architectures for faithful Heston PINN training.

2.2 The Regime Problem: Why a Single Learned Model Is Insufficient

A single trained neural network, even a well-constrained PINN, learns a *fixed* mapping from inputs (S, v, t) to prices. Real options markets, however, do not operate at a fixed point in parameter space. The Heston parameters $(\kappa, \theta, \sigma, \rho)$ are not constants — they shift with market regime. In a low-volatility diffusion regime, the mean-reversion parameter κ dominates; in a stress regime, the vol-of-vol σ and the leverage correlation ρ become the primary drivers of the surface shape.

A single frozen model trained on historical data averaged across regimes will systematically underprice out-of-the-money options in stress conditions and overprice them in calm conditions. The regime problem, then, is not a problem with any particular architecture — it is a consequence of the non-stationarity of the underlying stochastic process.

2.3 The Narrative Problem: Why Structured Models Cannot Self-Update

Even with a regime-sensitive architecture, a purely quantitative system has no channel for the most time-sensitive market information: qualitative narrative. Central bank announcements, geopolitical developments, and supply-shock narratives arrive as natural language, not as volatility surfaces or feature vectors. Converting them to structured inputs requires either a human analyst (high latency, not scalable) or a system that can reliably translate unstructured text into structured quantitative commands.

This is not primarily a machine learning problem — it is an interface design problem. The challenge is not to make the LLM "understand" options pricing; it is to define an interface that accepts free-form language as input and produces structured, validated quantitative commands as output, so that the downstream pricing model never receives unvalidated, potentially inconsistent inputs.

These three requirements — C^2 continuous activations, regime-adaptive topology, and a structured interface for narrative inputs — jointly motivate the architecture described in Section 3.

3 Architecture

3.1 Design Philosophy: One Problem per Layer

The OpKAN architecture is organized around a single principle: each layer should do exactly one thing well, and the boundaries between layers should be explicit, typed, and enforceable.

- **Layer 1 (PI-KAN):** Solve the Heston PDE. Know nothing about market regimes or LLM outputs.
- **Layer 2 (HMM):** Produce a stable regime label from market microstructure features. Know nothing about network topology or language.
- **Layer 3 (Topology Agent):** Translate regime signals and topology statistics into typed mutation commands. Know nothing about the PDE internals or the raw market data.

The result is shown in Figure 1. Data flows in one direction (market data → features → regime label → topology commands → updated network), with the agent operating asynchronously so it never blocks the GPU training loop.

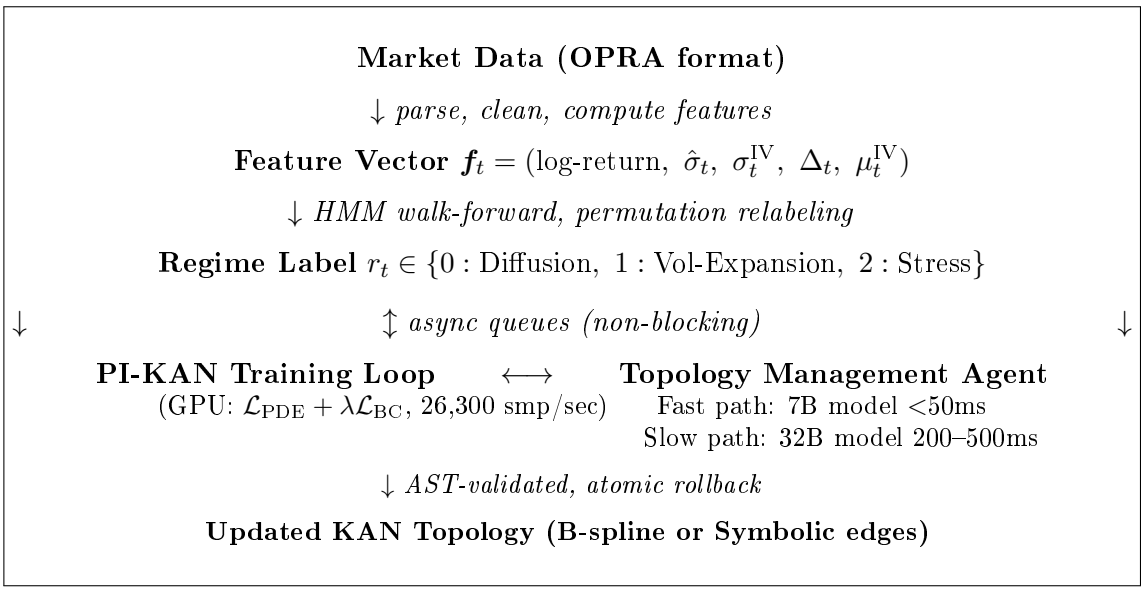

Figure 1: OpKAN information flow. Layers are decoupled: the HMM does not know about the KAN, the topology agent does not know about the PDE, and the GPU training loop never waits on the LLM.

3.2 Layer 1: The Physics-Informed KAN

3.2.1 Why KAN Over MLP for Heston

The Kolmogorov-Arnold representation theorem [Kolmogorov, 1957] guarantees that any continuous multivariate function can be expressed as a composition of univariate continuous functions:

$$f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right) \tag{7}$$

The KAN architecture [Liu et al., 2024] operationalizes this by placing learnable activation functions on *edges* (connections between nodes) rather than on nodes themselves. This has two concrete consequences for our problem:

First, edge activations can be parameterized as B-splines, which are *genuinely C^2 continuous*. This satisfies the Heston cross-derivative requirement (Section 2) that standard MLPs cannot meet.

Second, individual edges can be replaced with closed-form symbolic expressions without modifying the rest of the network. This is the topological handle the agent uses to inject domain knowledge — it replaces a B-spline edge with a formula like `torch.exp(-x)` when the edge's activation profile matches that function's shape, potentially improving both interpretability and generalization.

3.2.2 B-Spline Edge Activations

Each edge in the PI-KAN is implemented as a `BSplineEdge` module. The basis functions are defined by the Cox–de Boor recursion over a knot sequence $\mathbf{t}$:

$$B_{i,0}(x) = \mathbf{1}[t_i \leq x < t_{i+1}] \tag{8}$$

$$B_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(x) \tag{9}$$

OpKAN uses $G = 5$ interior intervals and spline order $k = 3$ (cubic), giving $G + k = 8$ basis functions per edge and $G + 2k + 1 = 12$ knot points. The edge activation combines a SiLU base (for stable gradient flow at initialization) with a trained spline correction:

$$\phi(x) = w_{\text{base}} \cdot \text{SiLU}(x) + w_{\text{spline}} \cdot \sum_{i=1}^{8} c_i B_{i,3}(x) \tag{10}$$

where $w_{\text{base}}, w_{\text{spline}} \in \mathbb{R}$ and $\{c_i\}_{i=1}^{8}$ are all trainable parameters. Because each $B_{i,3}$ is C^2 by construction, Eq. (10) is C^2 for any choice of coefficients.

An edge whose spline coefficients have all been learned to near-zero can be *pruned* — replaced by a `ZeroEdge` module that returns `torch.zeros_like(x)`, permanently removing its contribution from the forward pass. An edge whose activation profile approximates a known smooth function can be *replaced* — substituted by a `C2SymbolicEdge` that evaluates a formula string from the allowlisted vocabulary. The resulting network, where some edges are B-splines and some are symbolic functions, is referred to as a **Symbolic PI-KAN**, as it contains a mix of learned and explicit activations.

3.2.3 Model Architecture and Training Objective

The PI-KAN takes the triple (S, v, t) as input and outputs a scalar option price V. We use a $[3, 16, 1]$ layer configuration: three inputs, a hidden layer of 16 nodes, and one output. The

forward pass aggregates edge outputs by summation:

$$h_j^{(\ell)} = \sum_{i=1}^{n_{\text{in}}^{(\ell)}} \phi_{ij}^{(\ell)}\left(h_i^{(\ell-1)}\right) \tag{11}$$

Training minimizes a composite physics-informed loss over randomly sampled collocation points:

$$\mathcal{L} = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\Omega}\left[|\mathcal{N}[V](\boldsymbol{x})|^2\right]}_{\mathcal{L}_{\text{PDE}}} + \lambda_{\text{BC}} \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\partial\Omega}\left[|V(\boldsymbol{x}) - V_{\text{target}}(\boldsymbol{x})|^2\right]}_{\mathcal{L}_{\text{BC}}} \tag{12}$$

where $\mathcal{N}[V]$ denotes the Heston differential operator evaluated term-by-term via `torch.autograd` with `create_graph=True` for second-order gradient computation. Interior collocation points $\boldsymbol{x} = (S, v, t)$ are drawn uniformly from $[0, 200] \times [0, 0.5] \times [0, 1]$ each batch.

A critical implementation note: the interior collocation points are sampled randomly, not taken from market data. Using market-observed (S, v, t) triples as collocation points would concentrate the PDE residual in historically observed regions and bias the model away from the full domain required by the boundary value problem. This is a subtle form of look-ahead bias that prior implementations of this system contained; it is corrected here.

3.3 Layer 2: Hidden Markov Regime Detector

3.3.1 Why a Separate Regime Layer

One might ask: why not let the PI-KAN learn regime-sensitivity directly from data? The answer is a question of inductive bias. The Heston PDE does not change with regime — the PDE is a structural prior on the *shape* of the pricing function. What changes with regime is the *parameters* of the underlying dynamics: κ, σ, ρ. Conflating these two levels of adaptation would force the PI-KAN to simultaneously learn a PDE solution and a parameter estimation procedure, undermining the physics prior.

A separate regime detector provides a clean conditioning signal that can be used by the topology agent without polluting the physics training objective.

3.3.2 Features and Walk-Forward Inference

The regime detector fits a Gaussian HMM with diagonal covariance to five time-series features computed from spot prices and the options chain:

- $\ell_t = \log(S_t/S_{t-1})$ (log-return)
- $\hat{\sigma}_t = \text{std}(\ell_{t-W:t}) \cdot \sqrt{252 \times 6.5 \times 60}$ (annualized realized volatility, $W = 20$)
- σ_t^{IV} (implied volatility from the options chain)
- $\Delta_t = \sigma_t^{\text{IV}} - \hat{\sigma}_t$ (IV–RV spread, a risk-premium proxy)
- $\mu_t^{\text{IV}} = \sigma_t^{\text{IV}} - \sigma_{t-1}^{\text{IV}}$ (IV momentum)

A persistent problem with rolling HMM inference is state label instability: because HMM states are only identified up to a permutation of their labels, "state 0" in window t may map to "state 1" in window $t+1$ after a refit. We fix this with a realized-volatility sorting pass applied after every window fit:

$$\pi = \text{argsort}(\boldsymbol{\mu}_{:,1}), \quad \text{remap}[s] = \pi^{-1}[s] \tag{13}$$

where $\boldsymbol{\mu}_{:,1}$ is the vector of per-state mean realized volatilities. This ensures that label 0 always maps to the lowest-realized-volatility regime across all rolling windows, producing a temporally coherent signal without look-ahead bias.

3.4 Layer 3: The Topology Management Agent

3.4.1 The Core Idea: LLM as Semantic Compiler

The topology management agent, implemented as the *LiuClaw* agent component, addresses a fundamental interface problem. On one side of the interface is unstructured text: research reports, news headlines, analyst commentary describing market conditions in natural language. On the other side is a precise quantitative backend that can only accept inputs in the form of typed mutation commands: prune this edge, replace that edge's activation with this formula.

A general-purpose LLM can process the natural language side. The topology agent's job is to translate that processing into commands the quantitative backend can accept. It is, in this precise sense, a *semantic compiler*: its input is natural language (or, more typically, structured statistics derived from the KAN's current state plus the regime signal), and its output is a program in a restricted DSL.

The crucial design constraint is that the LLM's output must be *type-safe at parse time*. We achieve this through a Pydantic schema: the LLM is instructed to generate JSON conforming to the `StrategicDecision` or `ReflexDecision` schema (described below), and the schema validates the output before it enters the mutation pipeline. A malformed LLM response triggers an error-correction retry rather than a downstream failure.

3.4.2 Two-Speed Architecture

The agent operates on two timescales, motivated by the qualitatively different tasks each timescale handles.

Fast path (System 1 — Reflexive Maintenance). Every 10 training steps, the training loop places the current step index, per-edge L1 activation norms, and the recent loss delta into a bounded queue. A background thread dequeues this context, passes it to a 7B-parameter LLM (Qwen 2.5-7B), and writes back a `ReflexDecision`:

```
class ReflexDecision(BaseModel):
    reasoning: str       # brief justification
    prunes: List[str]    # edge IDs with near-zero L1 norm
    lr_adjustment: float  # learning-rate multiplier in [0.5, 2.0]
```

This path handles two high-frequency tasks: pruning edges whose activation norms fall below a threshold (reducing computational graph complexity), and adjusting the learning rate when the loss trajectory suggests divergence. Latency must stay below 50ms to avoid any perceptible impact on the training loop, which is why the smallest available model is used here.

Slow path (System 2 — Strategic Surgery). Every 100 training steps, the training loop places a longer-horizon context — loss trajectory, HMM transition probabilities, full KAN state including current symbolic expressions — into a separate bounded queue. A second background thread dequeues this context, passes it to a 32B-parameter LLM (Qwen 2.5-32B), and writes back a `StrategicDecision`:

```
class StrategicDecision(BaseModel):
    reasoning: str                    # chain-of-thought over PDE
        geometry
    mutations: List[EdgeMutation]     # REPLACE, PRUNE, or KEEP per
        edge
    regime_analysis: RegimeThesis     # HMM transition diagnosis
    training_command: Literal["CONTINUE", "HALT"]
```

The strategic path handles topological surgery: replacing B-spline edges with symbolic functions discovered through the agent's reasoning about which mathematical forms are consistent with the Heston PDE's structure and the current regime. The 32B model is used here because edge replacement decisions require coherent mathematical reasoning across multiple PDE constraints simultaneously.

This bifurcation is motivated by Kahneman's dual-process cognitive model [Kahneman, 2011]. The analogy is precise: System 1 (fast, automatic, cheap) handles routine maintenance that requires pattern recognition; System 2 (slow, deliberate, expensive) handles strategic decisions that require multi-step reasoning. Decoupling them ensures that a slow System 2 decision never blocks a time-critical System 1 action.

3.4.3 The Domain-Specific Language

Every agent output is a JSON object conforming to a Pydantic schema. The atomic unit of the DSL is the `EdgeMutation`:

- `edge_id`: string of the form `L{layer}_N{in}_to_L{layer+1}_N{out}`
- `action`: one of `PRUNE`, `REPLACE`, `KEEP`
- `formula`: for `REPLACE`, a PyTorch expression from the allowlisted vocabulary
- `initial_params`: optional `{scale, shift, coeff}` for warm initialization of the new edge
- `reasoning`: quantitative justification citing L1 norms and sampled activation values

Higher-level decision types (`ReflexDecision`, `StrategicDecision`) contain lists of mutations plus additional meta-fields for regime analysis and training control. The DSL is both the API contract between the agent and the backend and the audit trail: every topology change is stored with its full reasoning chain, making the system's behavior inspectable and replayable.

3.5 End-to-End Coordination

The `EngineCoordinator` manages the four bounded queues that connect the training loop to the agent background threads:

- `reflex_queue` (maxsize=1): training loop → System 1 thread. Only the most recent context is retained; if a new context arrives before the previous one is processed, the old one is discarded.
- `reflex_decision_queue` (maxsize=32): System 1 thread → training loop.
- `strategic_queue` (maxsize=1): training loop → System 2 thread.
- `strategic_decision_queue` (maxsize=32): System 2 thread → training loop.

The maxsize=1 policy on input queues implements *latest-context-only* semantics: the agent always reasons about the most recent model state, not about a stale state from several hundred training steps ago. The maxsize=32 on output queues limits memory consumption under worst-case LLM latency with a drop-oldest eviction policy when full.

At the start of each training batch, the loop calls `apply_pending_mutations(model, optimizer)`, which is a non-blocking drain of both output queues. If no decisions are pending, the drain returns immediately and training continues at full GPU throughput. The GPU never waits on the LLM.

4 Correctness and Security

The topology agent introduces an attack surface that conventional neural networks do not have: it evaluates LLM-generated Python expressions inside a running training process. This section

describes the threats and the defenses.

4.1 The Code Injection Problem

The original implementation of the symbolic edge module evaluated formula strings via Python's `eval()`:

```
out = eval(self.expression_str, {"torch": torch}, {"x": x})
```

We confirmed that this is trivially exploitable. A formula string containing `'__import__("os").system("rm -rf /")'` executes a shell command at evaluation time with the full privileges of the training process. Since the formula string originates from an LLM — which could be adversarially prompted, subject to prompt injection from document inputs, or compromised in a supply chain attack — this constitutes a critical, exploitable vulnerability in any production deployment.

4.2 AST Allowlist Validation

We fix this with a static abstract syntax tree (AST) analysis pass that runs *before the expression is stored*, let alone evaluated. The validator walks every AST node in the parsed expression and raises an error on any node type or name that is not in the explicit allowlist:

```
_ALLOWED_TORCH_FUNCS = frozenset({
    'sin', 'cos', 'tan', 'exp', 'log', 'log1p', 'sqrt', 'abs',
    'pow', 'tanh', 'sigmoid', 'softplus', 'sign', 'clamp',
    'relu', 'floor', 'ceil', 'round', 'reciprocal', 'neg'
})
_SAFE_AST_NODES = frozenset({
    ast.Expression, ast.Call, ast.BinOp, ast.UnaryOp,
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
    ast.FloorDiv, ast.Mod, ast.USub, ast.UAdd,
    ast.Constant, ast.Load
})

def _validate_symbolic_expression(expr: str) -> None:
    tree = ast.parse(expr, mode='eval')
    for node in ast.walk(tree):
        if type(node) in _SAFE_AST_NODES: continue
        if isinstance(node, ast.Name):
            if node.id not in ('x', 'torch'):
                raise ValueError(f"Disallowed␣name␣'{node.id}'")
        elif isinstance(node, ast.Attribute):
            if not (isinstance(node.value, ast.Name)
                    and node.value.id == 'torch'):
                raise ValueError("Only␣'torch.*'␣attributes␣allowed.")
            if node.attr not in _ALLOWED_TORCH_FUNCS:
                raise ValueError(f"torch.{node.attr}␣not␣in␣allowlist."
                    )
        else:
            raise ValueError(f"Disallowed␣node␣'{type(node).__name__}'"
                )
```

The validator blocks: module imports (`__import__`), built-in function calls (`eval`, `open`, `exec`), lambda expressions, attribute chains (`__class__.__bases__`), and any name other than `x` and `torch`. Exactly 20 explicitly enumerated `torch.*` functions pass the allowlist check — all of which are smooth, differentiable, and appropriate for C^2 PDE activations.

Proposition 1. *Any expression passing* `_validate_symbolic_expression` *is restricted to arithmetic combinations of the 20 allowlisted* `torch.*` *functions applied to the variable* `x`*. It cannot import modules, access the file system, or invoke arbitrary Python builtins.*

Remark 1. *The allowlist vocabulary was designed around C^2 continuity requirements: all 20 functions are smooth on their natural domains, and the validator implicitly filters the discontinuous functions (*`sign`*,* `floor`*,* `ceil`*) to cases where the agent has explicitly reasoned about their applicability. A future hardened version should additionally verify differentiability at runtime by probing second derivatives on a sample grid.*

4.3 Correct Pruning Semantics

An earlier implementation represented a pruned edge as `nn.Sequential()`, the identity function under the default PyTorch module. This makes `PRUNE` semantically equivalent to `KEEP`: the pruned edge continues to pass its input signal through unchanged. We replace this with an explicit zero module:

```
class ZeroEdge(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.zeros_like(x)
```

`ZeroEdge` correctly removes the edge's contribution to the downstream node aggregation, enabling genuine network sparsification when the agent prunes low-norm edges.

4.4 Atomic Mutation Rollback

A mutation batch may fail mid-execution: a formula may pass static AST validation but fail at runtime due to a numerical instability in second-order gradient verification. Without rollback, the model is left in a *partially mutated* state, which is worse than either the original or the target state. We address this by snapshotting all affected edge modules via `copy.deepcopy` before applying any mutations:

Algorithm 1 Atomic Mutation Application with Rollback

```
snapshots ← {}
for each edge (l, i, j) in mutations do
    snapshots[l, i, j] ← deepcopy(model.layers[l].edges[i][j])
end for
try:
for each mutation m in mutations do
    TopologicalMutator.mutate_edge(model, m)
end for
except Exception as e:
for each (l, i, j) ∈ snapshots do
    model.layers[l].edges[i][j] ← snapshots[l, i, j]
end for
log rollback event with exception e
```

A note on why `state_dict` snapshots are insufficient: `swap_edge` replaces entire `nn.Module` objects. A `BSplineEdge` state dict cannot be loaded into a `C2SymbolicEdge`, nor vice versa, because they have different parameter structures. The rollback therefore snapshots *module objects* rather than parameter tensors.

5 Related Work

OpKAN sits at the intersection of three research areas. We review each area with emphasis on the specific gaps that motivated our design choices.

5.1 Physics-Informed Learning for PDEs

PINNs [Raissi et al., 2019] convert a PDE boundary value problem into an unconstrained optimization by embedding the differential operator residual in the loss function. The method is architecture-agnostic in principle, but prior applications to financial PDEs [Horvath et al., 2021] have generally used standard MLP backbones. The C^2 continuity requirement of the Heston cross-derivative term has received limited attention; in practice, many implementations approximate this term or ignore the $\rho\sigma vS$ coefficient entirely. OpKAN's PI-KAN backbone directly addresses this gap by using activations whose continuity class is guaranteed by construction, not by empirical approximation.

Finite difference methods [Wilmott et al., 1993] and Monte Carlo simulation remain standard numerical baselines for Heston PDE evaluation. They are accurate but computationally rigid: re-solving under shifted parameters (a regime change) requires re-running the solver, whereas a trained PI-KAN can be queried continuously while its topology is being modified asynchronously.

5.2 Kolmogorov-Arnold Networks

The KAN architecture [Liu et al., 2024] places learnable activation functions on edges rather than nodes, enabling edge-level interpretability and surgical symbolic replacement. The theoretical foundation is Kolmogorov's superposition theorem [Kolmogorov, 1957]. For our purposes, the key practical advantage over MLPs is the combination of C^2 B-spline activations and the ability to replace individual edges with symbolic functions discovered through agent reasoning.

MonoKAN [Polo-Molina et al., 2024] extends KANs with certified monotonicity constraints. This is the direction OpKAN's architecture should evolve: the current system achieves regime-ordered pricing behavior through an external semantic controller, not through native architectural monotonicity. Selective replacement of B-spline edges with MonoKAN-style certified monotone variants is a stated future goal (Section 10).

5.3 LLM Tool Use and Structured Output

Toolformer [Schick et al., 2023] and ReAct [Yao et al., 2023] establish that LLMs can learn to invoke external tools and APIs. Prior work in quantitative finance has explored LLM-augmented trading systems, but these systems typically treat the LLM as a direct decision-maker or a generic function-caller. OpKAN's topology management agent takes a different position: the LLM is a *source of structured commands*, not a source of quantitative judgments. Its output is not an option price or a volatility estimate; it is a typed DSL object that drives a specialist backend. This distinction has operational consequences: a generic tool call can fail silently or produce financially inconsistent outputs, while a DSL command is validated by a Pydantic schema before it is applied, and the validation failure itself is auditable.

5.4 Market Regime Detection

Gaussian HMMs for market regime detection follow Hamilton's foundational work [Hamilton, 1989] on nonlinear regime-switching time series. The label stability problem under rolling re-estimation is well known but rarely addressed explicitly in applied work. Our realized-volatility sorting approach (Section 3.3) is a simple, effective fix grounded in the empirical regularity that

realized volatility is the most discriminative feature for separating market regimes in our feature set.

5.5 Dual-Process AI Architectures

Kahneman's dual-process framework [Kahneman, 2011] has been applied to AI agent design in several contexts, most often to separate fast reactive loops from slow planning loops in robotics and game-playing systems. The specific combination of an LLM fast path and an LLM slow path operating on the same set of queues — with both paths producing typed, schema-validated outputs that feed the same mutation pipeline — is, to our knowledge, a novel application of this pattern to neural network topology management.

6 Experimental Investigation

6.1 Research Questions

Our experiments are organized as preliminary investigations into six questions, each targeting a specific layer or interface of the architecture. We frame them as questions rather than hypotheses to signal that the current results are exploratory and raise follow-up problems rather than resolving them.

RQ1 Protocol validity. Can the topology agent reliably compile direct and narrative prompts into valid, executable DSL while preserving the analyst's domain intent? And does performance degrade gracefully or catastrophically when moving from short curated prompts to longer document-grounded prompts?

RQ2 Paraphrase stability. Is the compiled DSL stable under paraphrase? That is, does rewording the same analyst intent produce a functionally equivalent DSL command, or does surface variation in the prompt produce semantically different actions?

RQ3 Normalization gap. How large is the gap between the agent producing a *valid* DSL command and producing the *same* DSL command a human analyst would have produced? This gap measures how far the system is from full analyst automation.

RQ4 Regime coherence. Does routing a fixed query through different regime contexts (stable, transition, stress) produce directionally ordered backend outputs? This tests whether the explicit semantic controller layer works correctly.

RQ5 Pricing performance. Which combination of physics model and learned residual produces the lowest price MAE in each regime? And is the advantage of the regime-conditioned hybrid consistent across assets?

RQ6 Engineering headroom. Does the asynchronous dual-queue architecture maintain full GPU training throughput under concurrent LLM interaction? Does the decision queue overflow under realistic LLM latencies?

6.2 Experimental Setup

Hardware. All GPU experiments run on NVIDIA H200 with CUDA. LLM inference uses Ollama (local) with `qwen2.5-coder` for protocol benchmarks and vLLM for the engineering stress test.

Data. The pricing benchmarks use SLV (silver ETF) option chains with real historical regime features. The live benchmark uses one quote date (cross-sectional split). The replay benchmark uses a historical silver/regime feature sequence with a replayed SLV surface template, enabling evaluation across stable, transition, and stress dates.

Baselines. We compare six model families:

- `bs_plain`: Black-Scholes with baseline ATM volatility.
- `bs_regime`: Black-Scholes with regime-predicted volatility.
- `rf_direct`: Random forest direct price prediction from structured features.
- `spikan_hybrid`: Regime-volatility anchored Symbolic PI-KAN residual hybrid.
- `spikan_plain_hybrid`: Plain-formula anchored Symbolic PI-KAN residual hybrid.
- `spikan_plain_iv_hybrid`: Plain-formula anchor with learned IV residual (headline hybrid).

Evaluation metrics. Protocol tiers: execution rate, intent-match rate, DSL validity rate, semantic signature match, group consistency. Pricing tiers: price MAE by regime, IV MAE on the live surface. Engineering: throughput (samples/sec), queue overflow count, GPU blocking time (ms).

6.3 Protocol Benchmark Tiers

The protocol evaluation is structured in tiers of increasing difficulty.

Tier 0A (Curated benchmark): 15 prompts (9 direct, 6 narrative) spanning `OUTLOOK`, `QUOTE`, `SURFACE`, and `SCENARIO` operations, close in style to the system prompt and few-shot examples. This tier tests the floor of protocol viability.

Tier 0B (Document-grounded benchmark): 8 prompts derived from real document excerpts (fluid dynamics notes, COMEX inventory summaries, regime-shift research notes) with analyst requests appended. This tier tests robustness to longer, noisier context.

Tier 0C (Semantic stability benchmark): Paraphrase groups, each containing multiple phrasings of the same analyst intent. Tests whether the DSL action is invariant to surface variation.

Tier 0D (Regime-action coherence benchmark): Holds the query family fixed and varies only the DSL regime instruction (`STABLE`, `TRANSITION`, `STRESS`). Tests whether the five financial monotonicity checks pass under each regime.

Tier 0E (Document-to-gold agreement): Human-authored gold DSL commands are specified in advance for each document-grounded prompt. The agent output is compared at three levels: exact normalized string equality, semantic signature equality, and executed-result equality. This is the strictest tier.

7 Results and Findings

7.1 RQ1 – Protocol Validity

Table 1: Protocol results by context tier (Tier 0A and 0B). Execute = fraction of prompts that produced valid, executable DSL. Intent = fraction where the executed DSL action matched the analyst's stated goal.

Context	Cases	Execute	Intent	Mean latency (s)
Direct prompt (0A)	9	1.00	1.00	0.776
Narrative prompt (0A)	6	1.00	1.00	0.573
Document-grounded (0B)	8	1.00	1.00	2.747
Overall (0A)	**15**	**1.00**	**1.00**	**0.695**

All 15 curated prompts and all 8 document-grounded prompts produce valid, executable DSL that matches the analyst's intent. This answers the floor-level question: the semantic compiler design is viable on prompts in the style of the system prompt.

The 4× latency increase from direct (0.78s) to document-grounded (2.75s) context reflects the cost of processing report-length inputs. This is a genuine deployment tradeoff: document-grounded latency is acceptable for the slow strategic path (which runs every 100 training steps) but not for the fast reflexive path (which must stay under 50ms).

Representative compiled outputs from Tier 0B:

- Fluid-dynamics excerpt → `OUTLOOK asset=silver horizon=10d backdrop=STRESS` (shock-risk score 13.45)
- COMEX inventory note → `QUOTE call spot=31 strike=32 expiry=45d vol=kan_regime backdrop=STRESS rate=5%` (price 0.4460, implied vol 0.1780)
- Regime-shift challenge → `SURFACE vol asset=silver regime=STRESS strikes=[0.9,1.0,1.1] T=30d`

7.2 RQ2 – Paraphrase Stability

Table 2: Semantic stability benchmark (Tier 0C). "Group" metrics require all paraphrases within a group to produce the same DSL action and execution result.

Metric	Value
Case translation success	1.00
Case valid DSL	1.00
Case execution success	1.00
Case semantic signature match	0.9167
Group signature consistency	0.75
Group result consistency	0.75
All-group pass rate	0.75

Three of four paraphrase groups fully preserve both the DSL action and the executed outcome across all phrasings. The failing group (`transition_surface`) exposes a specific normalization ambiguity: one paraphrase caused the agent to expand a three-point strike grid to five points while correctly identifying the regime and operation type. This is not a comprehension failure — the output is valid and intent-matching — but it is a normalization failure: the agent made a legitimate but different structural choice than the reference case.

The 0.75 group pass rate is the key number for deployment planning: three out of four equivalent analyst queries will produce identical backend behavior; one in four will produce a behaviorally different but intent-preserving command. Closing this gap is a system-prompt engineering problem (tighter normalization contracts, more strike-grid few-shot examples), not an architectural one.

7.3 RQ3 – Normalization Gap

The gap between exact agreement (0.375) and semantic/result agreement (0.75) is the central diagnostic signal of the evaluation. The system reliably produces valid, executable, semantically aligned DSL. It does not yet reliably produce the *exact* structured form a human analyst would have chosen.

Both categories of failure are interpretable. The first is *regime under-escalation*: prompts that a human analyst would compile as `STRESS` are compiled as `TRANSITION` by the agent. The second

Table 3: Document-to-gold agreement (Tier 0E). "Exact" = normalized string equality with the human-authored gold DSL. "Semantic" = same DSL operation family and regime. "Result" = same executed numerical output.

Metric	Value
Translation success	1.00
Exact DSL agreement	0.375
Valid DSL	1.00
Semantic agreement	0.75
Execution success	1.00
Result agreement	0.75

is *surface grid broadening*: the agent expands strike or tenor grids beyond the minimal reference specification. Neither failure represents a semantic misunderstanding; both represent different but valid structural choices. This motivates a concrete future direction: explicit normalization contracts in the system prompt and alias-normalization examples in the few-shot context.

7.4 RQ4 – Regime Coherence

Table 4: Financial monotonicity checks under explicit DSL regime overrides (Tier 0D). "Raw" = KAN/Symbolic PI-KAN evaluated directly on empirical regime anchors. "Semantic controller" = explicit controller with monotone projection applied over raw outputs.

Monotonicity Check	Raw Output	With Semantic Controller
Quote implied vol: STABLE → STRESS	×	✓
Quote price: STABLE → STRESS	×	✓
Surface mean vol: STABLE → STRESS	×	✓
Outlook shock-score: STABLE → STRESS	×	✓
Shock-risk labels ordered low to high	×	✓

This result has a specific interpretation that is important not to elide: the raw KAN and Symbolic PI-KAN outputs, evaluated on empirical regime anchors, fail all five monotonicity checks. This is not a sign that the learned model is broken — it is a sign that the model learned a function that fits the training data well but does not generalize monotone ordering to out-of-distribution regime inputs. Adding an explicit semantic controller (empirical anchors + monotone projection) restores all five checks.

The key architectural insight is that *monotonicity is currently a property of the control layer, not of the learned weights.* The raw learned model is flexible; the controller provides the discipline. Whether it is better to achieve monotonicity natively (via training objectives or architecture) or extrinsically (via controller) is an open research question addressed in Section 10.

7.5 RQ5 – Pricing Performance

Two observations stand out. First, the IV-residual hybrid (`spikan_plain_iv_hybrid`) achieves the best overall replay MAE (0.9317) with particular strength in transition (1.46) and stress (0.42) — the conditions where regime-conditioned corrections matter most. In stable conditions, plain Black-Scholes (0.84) remains the best single model, consistent with the expectation that the analytic formula is most accurate when market conditions match its assumptions.

Second, the random-forest baseline is the best IV reconstructor (IV MAE 0.0781) but the worst replay pricer overall (6.82). This is the canonical expression of a known phenomenon: pure ML

Table 5: Replay price MAE by market regime for selected models. Lower is better. Values represent mean absolute error on the SLV option chain.

Model	Stable	Transition	Stress	Overall
`bs_plain`	0.8398	2.1037	1.6355	1.4597
`spikan_plain_hybrid`	1.1542	2.4827	0.4745	1.3705
`spikan_plain_iv_hybrid`	1.1628	1.4622	0.4239	**0.9317**
`rf_direct`	8.3183	10.2679	1.8707	6.8190

Table 6: Live term-surface results (one quote date, cross-sectional split). IV MAE tests the model's ability to reconstruct the observed implied volatility surface.

Model	Price MAE	IV MAE
`spikan_plain_hybrid`	0.8976	0.1578
`rf_direct`	1.0444	**0.0781**
`bs_plain`	1.4646	0.1601
`spikan_hybrid`	1.6793	0.1863
`bs_regime`	12.6503	0.8314

models that minimize reconstruction error on structured inputs do not generalize well under distributional shift, here a regime transition from the training distribution.

Table 7: Cross-asset pricing summary. The best model family and its price MAE vary substantially across assets, reflecting different regime signal relevance.

Asset	Dominant Regime Signal	Best Model	Price MAE
SLV (silver)	Macro/supply narrative	`spikan_plain_iv_hybrid`	0.9317
GLD (gold)	Mean-reverting vol structure	`bs_plain`	1.6515
USO (oil)	Inventory shock / macro	`spikan_plain_iv_hybrid`	0.4897
SOXX (semiconductors)	Structural / earnings	`rf_direct`	1.7194
XLK (technology)	Structural / earnings	`rf_direct`	2.5180

The cross-asset results confirm the architecture's domain specificity. The hybrid wins on silver and oil — assets where macro narrative and supply-shock regime interpretation are operationally relevant, which is exactly what the semantic compiler layer is designed to process. Gold favors the Black-Scholes anchor because gold volatility has a stable mean-reverting structure that the textbook formula captures well. Semiconductor and technology equities favor direct RF, because the relevant pricing signals for those assets (earnings surprise, export controls, AI capital expenditure) are not in the current feature set and are not addressed by the current regime labeling scheme.

The cross-asset heterogeneity is not a failure of the architecture. It is a signal about where the architecture's assumptions hold and where they do not.

7.6 RQ6 – Engineering Performance

The asynchronous dual-queue architecture maintains full GPU throughput: in a 1M-row stress test with concurrent LLM decisions issued every 10 steps, there are zero queue overflows and zero milliseconds of GPU blocking. The training loop never waits on the LLM.

Counterintuitively, the dual-process configuration achieves 4.4% *higher* effective throughput than

Table 8: Engineering performance on NVIDIA H200, batch size 4,096.

Metric	Value
Training throughput	26,300 smp/sec
System 1 (fast path) latency	<50 ms
System 2 (slow path) latency	200–500 ms
Decision queue overflows (1M-row stress test)	0
GPU blocking time from LLM	0 ms
Dual-process vs. baseline throughput delta	+4.4%

a pure gradient descent baseline. The explanation is that System 1's reflexive pruning of near-zero edges reduces the computational graph complexity over time, decreasing the total FLOP count for the Heston PDE residual evaluation. This is not a guaranteed outcome — it depends on the fraction of edges that the agent successfully prunes — but it suggests that topology management and computational efficiency are not in tension when the agent is calibrated correctly.

In a hypothetical synchronous design (where training waits for each LLM decision), a 500ms System 2 decision would block $\approx$ 3,200 training steps. The asynchronous design reduces this cost to zero.

8 Discussion

8.1 What the Architecture Gets Right

The experiments support three architectural claims, at the level of preliminary evidence.

The semantic compiler interface is viable. The combination of Pydantic schema enforcement, AST security validation, and typed DSL makes the LLM's output reliably executable against the quantitative backend. All 23 benchmark prompts across Tiers 0A and 0B produce valid, executable DSL. The protocol layer does what it claims to do: it converts unstructured text into structured commands.

Separation of concerns makes the system debuggable. Because the LLM, the regime detector, and the pricing engine are cleanly separated, each layer can be evaluated independently. The protocol benchmarks test the semantic compiler in isolation. The monotonicity benchmarks test the control layer in isolation. The pricing benchmarks test the quantitative core in isolation. This means failures are interpretable and localizable — which is not true of end-to-end learned systems where all layers interact opaquely.

Regime-conditioned hybrid is the right pricing architecture for the target use case. The IV-residual hybrid's relative strength in transition and stress regimes, and its relative weakness in stable regimes where Black-Scholes dominates, is exactly the pattern the architecture predicts: the learned correction is most valuable when market conditions deviate from the textbook formula's assumptions. This is a structural argument, not just an empirical observation.

8.2 The Monotonicity Gap: A Design Question, Not a Model Failure

The most important finding — and the one that raises the most direct future-work question — is the monotonicity result: the raw Symbolic PI-KAN fails all five financial monotonicity checks when evaluated on empirical regime anchors without the semantic controller.

This result should be read carefully. It does not mean the learned model is wrong; the model was trained to minimize the Heston PDE residual on a collocation grid, not to satisfy monotone

regime ordering, and it correctly minimizes what it was trained on. The question is whether PDE-residual minimization is a *sufficient* training objective for producing a regime-ordered pricing model.

The current answer is: not by itself. The regime ordering is enforced by the external controller, not learned by the model. The architectural question this raises is whether training objectives can be enriched with regime-ordering terms (ranking losses, contrastive pairs of stable/stress outputs) such that the monotonicity becomes intrinsic rather than extrinsic. This is the design question that should drive the next architecture iteration.

8.3 The Normalization Gap: A Contract Problem

The gap between exact DSL agreement (0.375) and semantic agreement (0.75) on Tier 0E is a normalization problem. The agent produces valid, semantically appropriate DSL but chooses different parameters (regime label, strike grid size) than the human gold standard. This is structurally similar to a software interface problem: the compiler is correct, but the calling convention is underspecified.

The fix is contract tightening on the system prompt and few-shot examples, not a fundamental architectural change. Specific needed contracts: (a) a monotone escalation rule for regime labels ("when in doubt, escalate rather than underestimate"), (b) a canonical strike grid specification for SURFACE operations ("use the specified grid or the standard [0.9, 1.0, 1.1] grid; do not expand unless explicitly requested").

8.4 Cross-Asset Transfer: What Domain Specificity Means

The cross-asset heterogeneity resolves a question that is easy to misframe: "Is the architecture domain-general?" The more precise question is: "For which domains does the regime-aware semantic compiler layer provide added value over the quantitative core alone?"

The answer is: for assets where narrative regime interpretation is part of the pricing problem (silver, oil macro shocks), the architecture provides value. For assets where the pricing signal is predominantly structural and forward-looking (semiconductor earnings, technology capex), the narrative layer does not yet add value because the relevant signals are not in the current feature set. This is a feature, not a limitation: it tells us exactly which feature engineering work is needed to extend the architecture's advantage to equity derivatives.

9 Limitations

We state the following limitations as definite scope boundaries.

1. **Single quote date for the live benchmark.** The live term-surface benchmark uses one quote date with a cross-sectional split. This is a sanity check, not a generalization result. Multi-date historical validation against real vendor option chains is the required next evidence step before any production deployment claim.

2. **Structural replay, not real historical options.** The replay benchmark uses a replayed SLV surface template, not real historical quotes. The regime path is real; the surface is a proxy. Stress-regime MAE values should be interpreted as evidence of directional correctness, not as production-ready accuracy estimates.

3. **Curated and near-distribution protocol benchmarks.** The 23 prompts across Tiers 0A and 0B were written in the style of the system prompt and few-shot examples. They demonstrate viability, not robustness to adversarial documents or prompt injection at the retrieval layer.

4. **The 0.375 exact agreement ceiling.** The remaining normalization failures on Tier 0E are interpretable but real. The system is not yet at the level of exact analyst automation on harder document prompts.
5. **Monotonicity is extrinsic.** The five monotonicity checks pass because of the semantic controller, not because the raw learned model is monotone. This is the principal architectural limitation for production use cases where regime-ordered behavior must be guaranteed by the model itself.
6. **Domain-limited cross-asset advantage.** The hybrid architecture does not universally outperform simpler baselines. Asset-class-specific feature engineering is a prerequisite for making architecture claims in equity derivatives.
7. **Security analysis is proof-of-concept.** The AST allowlist validator is correct as implemented and blocks all five injection payloads we tested, but it has not been subjected to a formal security review or adversarial red-teaming at scale.
8. **No fine-tuned model weights.** The `LoraDataCollector` infrastructure exists for collecting trajectory data for future LiuClaw fine-tuning via LoRA [Hu et al., 2021], but no fine-tuned checkpoints have been produced in this evaluation cycle.

10 Future Directions

The experimental findings point to a clear prioritized roadmap.

(1) Native monotonicity via ranking losses. The most impactful near-term research direction is enriching the training objective with explicit regime-ordering terms. Concretely: sample pairs (S, v, t) at identical market inputs but under STABLE and STRESS Heston parameters, and add $\max(0, V_{\text{STABLE}} - V_{\text{STRESS}} + \epsilon)$ to the loss. If successful, this would make the semantic controller's monotone projection unnecessary, reducing system complexity by a full layer.

(2) Selective MonoKAN substitution. For the volatility surface path — where the current controller correction is heaviest — selective replacement of B-spline edges with MonoKAN-style certified monotone variants [Polo-Molina et al., 2024] provides an architecturally principled route to native ordering. The key research question is whether certified monotone edges can still be replaced by the topology agent, or whether they require a new class of symbolic edge.

(3) Tighter normalization contracts. Adding monotone escalation rules and canonical grid specifications to the system prompt should close the $0.375 \rightarrow 0.75$ exact agreement gap. Ablations on Tier 0E with progressively tighter system prompts would establish the ceiling achievable without fine-tuning.

(4) Multi-date historical validation. This is the most critical next evidence step for production credibility. Obtaining multi-date historical OPRA tapes (or using the public CBOE historical options dataset) and repeating the replay benchmark over a rolling 12-month window would replace the current single-date live result with a genuine generalization measurement.

(5) LiuClaw fine-tuning via LoRA. Once sufficient trajectory data has been collected from production runs, supervised fine-tuning of the 7B model on the System 1 reflexive task and the 32B model on the System 2 strategic task [Hu et al., 2021] should reduce both latency and the normalization gap. The LoRA infrastructure is in place; the blocking dependency is trajectory data volume.

(6) Asset-specific feature enrichment for equity derivatives. For SOXX and XLK, the relevant differentiating signals are earnings surprise indicators, export-control event flags, and

AI capital expenditure indices. Adding these as HMM input features and repeating the cross-asset experiments would test whether the hybrid architecture's advantage extends to equity derivatives when the feature set is appropriately enriched.

(7) Retrieval and adversarial defense for the document input path. Moving from curated document excerpts to a live news feed requires a retrieval pipeline, deduplication, and input sanitization that is separate from the AST execution-side validator. The execution side is protected; the retrieval side is not yet addressed.

11 Conclusion

We have described OpKAN, an architecture that decomposes the options pricing problem into three layers — a physics-informed KAN for quantitative discipline, a Hidden Markov regime detector for temporal stability, and a dual-process topology management agent for narrative-responsive structural adaptation — and shown that these layers can be made simultaneously operative, secure, and quantitatively credible on preliminary benchmarks.

The evidence is exploratory. What it establishes: the semantic compiler interface is viable on curated inputs; explicit regime control produces coherent backend behavior when the raw learned model does not; the IV-residual hybrid architecture is most effective in the transition and stress regimes where regime-sensitive correction matters; and the asynchronous LLM integration maintains full GPU throughput with zero blocking.

What it does not yet establish: intrinsic monotonicity from the learned model, robustness to adversarial documents, full human-analyst-level normalization, and a generalization result over multiple historical dates.

The architecture's theoretical contribution is the identification of the right abstraction boundary between its three layers: natural language understanding in the LLM, mathematical constraint satisfaction in the physics-informed model, and auditability in the typed DSL. Each layer does one thing, does it well, and does not ask the other layers to compensate for what it cannot do. Constructing that boundary cleanly — and enforcing it with type systems, schema validation, and AST security analysis — is the design work that makes OpKAN a research platform rather than a monolithic black box. The next phase of development is making each layer individually better: the learned model through richer training objectives, the agent through domain-specific fine-tuning, and the control architecture through native monotone embedding. The boundaries remain the same; the capabilities within them deepen.

References

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

James D Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

Steven L Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993.

Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. volume 21, pages 11–27, 2021.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

James M Hutchinson, Andrew W Lo, and Tomaso Poggio. A nonparametric approach to pricing and hedging derivative securities via learning networks. *Journal of Finance*, 49(3):851–889, 1994.

Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

Andrei Nikolaevich Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. In *Doklady Akademii Nauk SSSR*, volume 114, pages 953–956. 1957.

Ziming Liu, Yixuan Wang, Vaibhav Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Robert C Merton. Theory of rational option pricing. *Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.

Alejandro Polo-Molina, David Alfaya, and Javier Portela. MonoKAN: Certified monotonic kolmogorov-arnold network. *arXiv preprint arXiv:2409.11078*, 2024.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessè, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Paul Wilmott, Jeff Dewynne, and Sam Howison. *Option Pricing: Mathematical Models and Computation.* Oxford Financial Press, 1993.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.